

Spring Events & Integration

Event-driven architecture, messaging, and scheduling in Spring

Sub-Chapter 03-09 | Beginner-Friendly | With Diagrams & Q&A;

Spring Application Events	Publish custom events with <code>ApplicationEvent</code> & <code>ApplicationEventPublisher</code>
@EventListener	Listen to events with a simple annotation on any method
Async Events	<code>@Async</code> + <code>@EnableAsync</code> : run listeners on a background thread pool
Built-in Spring Events	<code>ContextRefreshedEvent</code> , <code>ContextClosedEvent</code> & lifecycle hooks
Spring Integration Overview	Channels, endpoints, and message flow for enterprise integration
@MessagingGateway & @ServiceActivator	Simplest way to send & receive messages in Spring Integration
Spring Scheduling	<code>@Scheduled</code> + <code>@EnableScheduling</code> : <code>cron</code> , <code>fixedRate</code> , <code>fixedDelay</code>

1 Spring Application Events

ApplicationEvent + ApplicationEventPublisher

Overview

Spring has a built-in **event system** based on the Observer pattern. Any bean can **publish** an event via **ApplicationEventPublisher** and any other bean can **listen** for it. The ApplicationContext itself acts as the event bus -- it routes each event to every listener that declared interest in that event type. This decouples producers from consumers: neither needs to know about the other.

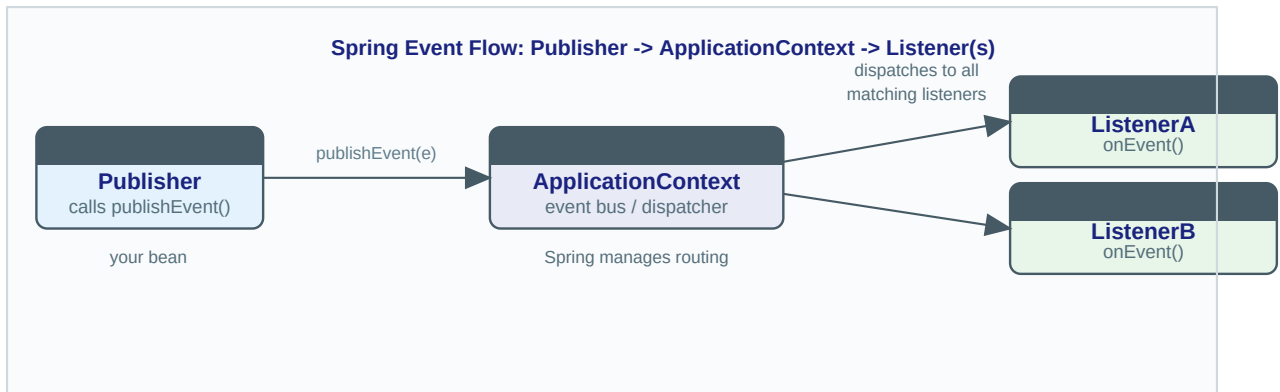


Figure: event flows from publisher through ApplicationContext to all matching listeners

Simple Java Example

```
// 1. Define the event
public class OrderPlacedEvent extends ApplicationEvent {
    private final String orderId;
    public OrderPlacedEvent(Object src, String orderId) {
        super(src); this.orderId = orderId;
    }
    public String getOrderId() { return orderId; }
}

// 2. Publish it
@Service
public class OrderService {
    @Autowired ApplicationEventPublisher publisher;

    public void placeOrder(String id) {
        publisher.publishEvent(new OrderPlacedEvent(this, id));
    }
}

// 3. Listen (implements ApplicationListener)
@Component
public class EmailNotifier
    implements ApplicationListener<OrderPlacedEvent> {
    public void onApplicationEvent(OrderPlacedEvent e) {
        System.out.println("Send email for: " + e.getOrderId());
    }
}
```

Key Rules

- Extend `ApplicationEvent` to create a custom event type.

- Inject `ApplicationEventPublisher` and call `publishEvent()` to fire the event.
- Implement `ApplicationListener` to receive it -- or use `@EventListener` (Section 2).
- By default events are synchronous: the publisher thread calls all listeners in order.
- Events are scoped to the `ApplicationContext`; child contexts do not see parent events by default.

Q1. What is `ApplicationEventPublisher`?

A Spring interface injected into any bean that lets it fire events to the `ApplicationContext`, which then routes them to all registered listeners.

Q2. Must an event class extend `ApplicationEvent`?

In Spring 4.2+ you can publish any plain object as an event -- Spring wraps it automatically. Extending `ApplicationEvent` is the traditional, explicit way.

Q3. Are events synchronous by default?

Yes. The publisher thread calls each listener sequentially before `publishEvent()` returns, unless `@Async` is used on the listener (see Section 3).

Q4. How many listeners can receive one event?

Unlimited. Every bean that implements `ApplicationListener` or has an `@EventListener` method for that type will receive it.

Q5. Can a listener also publish events?

Yes -- a listener can inject `ApplicationEventPublisher` and fire further events, enabling event chains. Be careful to avoid infinite loops.

2 @EventListener

Listen to events with a simple annotation

Overview

@EventListener is the annotation-based alternative to implementing `ApplicationListener`. Place it on any public method in a managed bean; Spring detects the event type from the method parameter and calls the method whenever that event is published. You can also add a **condition** using SpEL to filter events, and a method can listen to multiple event types at once.

Simple Java Example

```
@Component
public class NotificationService {

    // Basic listener -- fires for every OrderPlacedEvent
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        System.out.println("Order: " + event.getOrderId());
    }

    // Conditional -- only if orderId starts with "VIP"
    @EventListener(condition = "#event.orderId.startsWith('VIP')")
    public void onVipOrder(OrderPlacedEvent event) {
        System.out.println("VIP order: " + event.getOrderId());
    }

    // Listen to multiple types
    @EventListener({OrderPlacedEvent.class, OrderCancelledEvent.class})
    public void onOrderChange(ApplicationEvent event) {
        System.out.println("Order changed");
    }
}
```

Key Rules

- **@EventListener** infers the event type from the method's parameter type.
- Use `condition = "#event.field == value"` (SpEL) to filter events without if-statements.
- A method can declare multiple event types in `@EventListener({A.class, B.class})`.
- The method can return a value -- Spring publishes the return value as a new event.
- **@EventListener** requires the bean to be managed by Spring (`@Component`, `@Service`, etc.).

Q1. How does **@EventListener** know which event to listen to?

It inspects the method's parameter type. If the parameter is `OrderPlacedEvent`, the method is called only when that event type (or a subtype) is published.

Q2. Can I filter events with **@EventListener**?

Yes -- add `condition = "#event.someField == 'value'"` using Spring Expression Language (SpEL). The method is skipped if the condition is false.

Q3. What happens if an **@EventListener** method returns a non-void value?

Spring automatically publishes the returned object as a new event, enabling event chaining.

Q4. Does **@EventListener** work inside **@Configuration** classes?

Yes, but it is cleaner to put listeners in `@Component` or `@Service` classes.

Q5. What is the difference between @EventListener and ApplicationListener?

@EventListener needs less boilerplate -- no interface to implement, works on any method, and supports SpEL conditions. Both are functionally equivalent for simple use cases.

3 Async Events

@Async + @EnableAsync: non-blocking listener execution

Overview

By default Spring dispatches events **synchronously** -- the publisher thread calls every listener before `publishEvent()` returns. For slow listeners (sending email, calling a remote API) this blocks the caller unnecessarily. Adding **@Async** to a listener method makes Spring hand it off to a thread pool and return immediately. You must also enable async support with **@EnableAsync** on a configuration class.

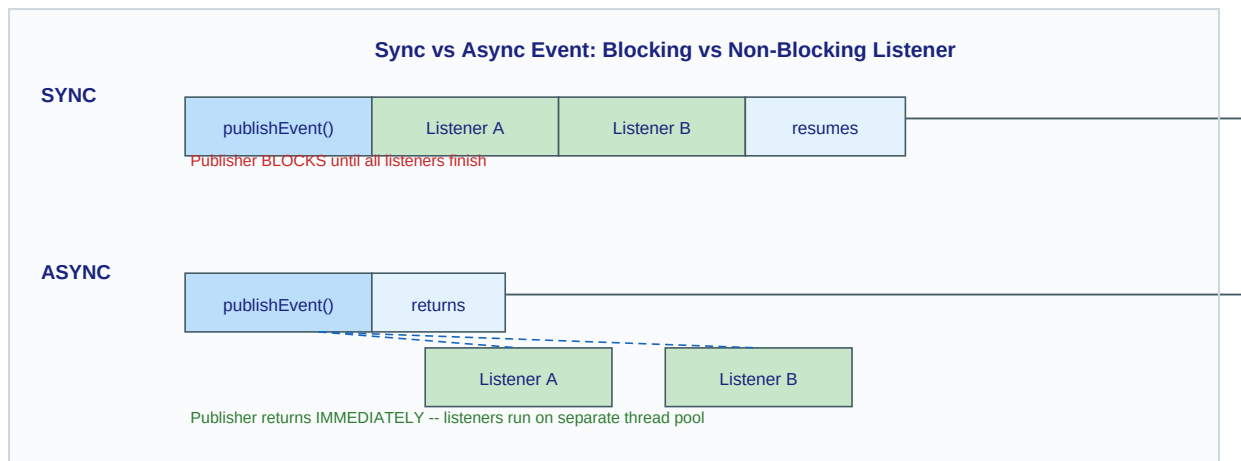


Figure: sync listener blocks the publisher; async listener runs on a separate thread

Simple Java Example

```
// Step 1: enable async support
@Configuration
@EnableAsync
public class AsyncConfig { }

// Step 2: mark the listener as async
@Component
public class SlowEmailSender {

    @Async
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        sendEmail(event.getId()); // runs on thread pool
    }
}

// Step 3 (optional): custom thread pool
@Bean
public TaskExecutor asyncExecutor() {
    ThreadPoolTaskExecutor ex = new ThreadPoolTaskExecutor();
    ex.setCorePoolSize(4);
    ex.setMaxPoolSize(10);
    ex.initialize();
    return ex;
}
```

Key Rules

- Add **@EnableAsync** to a **@Configuration** class to activate async processing.

- @Async on a listener method makes it run on Spring's SimpleAsyncTaskExecutor by default.
- Define a ThreadPoolTaskExecutor bean for production -- the default creates threads unboundedly.
- Exceptions thrown by async listeners are NOT propagated back to the publisher.
- @Async and @EventListener order: put @Async ABOVE @EventListener on the method.

Q1. Why would you make an event listener async?

To avoid blocking the publisher thread on slow operations like sending email, writing logs to a remote system, or calling external APIs.

Q2. What annotation enables async support in Spring?

@EnableAsync on any @Configuration class. Without it, @Async annotations are silently ignored.

Q3. What thread pool does @Async use by default?

Spring uses SimpleAsyncTaskExecutor, which creates a new thread per task. For production, define a ThreadPoolTaskExecutor bean named 'taskExecutor'.

Q4. Can an async listener return a value?

Not directly. It can return Future or CompletableFuture, but event chaining (returning a new event) does not work with async listeners.

Q5. What happens if an async listener throws an exception?

The exception is swallowed by default (logged by the task executor). The publisher never sees it. Configure an AsyncUncaughtExceptionHandler to handle these errors.

4 Built-in Spring Events

ContextRefreshedEvent, ContextClosedEvent & lifecycle hooks

Overview

Spring publishes several **built-in application events** at different points in the `ApplicationContext` lifecycle. You can listen to these the same way you listen to custom events -- with `@EventListener` or `ApplicationListener`. They are useful for running code at startup, detecting context reloads, or cleaning up resources on shutdown.

Built-in Events Reference

Event Class	When Published	Common Use
<code>ContextRefreshedEvent</code>	After <code>ApplicationContext</code> is initialized or refreshed (all beans ready)	Warm up caches, validate configuration
<code>ContextStartedEvent</code>	When <code>ApplicationContext.start()</code> is called explicitly	Resume paused components
<code>ContextStoppedEvent</code>	When <code>ApplicationContext.stop()</code> is called explicitly	Pause background tasks
<code>ContextClosedEvent</code>	When <code>ApplicationContext.close()</code> is called (JVM shutdown or destroy)	Release resources, close connections
<code>ApplicationReadyEvent</code>	After the app is ready to service requests (Spring Boot only)	Trigger startup logic after server is up
<code>ApplicationFailedEvent</code>	If the application fails to start (Spring Boot only)	Alert/cleanup on startup error

Simple Java Example

```
@Component
public class StartupRunner {

    // Runs after all beans are initialized
    @EventListener(ContextRefreshedEvent.class)
    public void onContextReady() {
        System.out.println("Context ready -- warming up cache");
    }

    // Spring Boot: runs after server is accepting requests
    @EventListener(ApplicationReadyEvent.class)
    public void onAppReady() {
        System.out.println("App is live!");
    }

    // Clean up on shutdown
    @EventListener(ContextClosedEvent.class)
    public void onShutdown() {
        System.out.println("Shutting down -- releasing resources");
    }
}
```


Key Rules

- ContextRefreshedEvent fires every refresh -- guard idempotent startup code with a flag.
- Prefer ApplicationReadyEvent over ContextRefreshedEvent in Spring Boot apps.
- ContextClosedEvent is reliable for cleanup; alternatively use @PreDestroy on a bean.
- Built-in events follow the same routing as custom events -- all listeners are notified.
- In a parent/child context setup, events bubble up to the parent context.

Q1. When is ContextRefreshedEvent published?

After the ApplicationContext finishes initializing all beans, and again every time the context is refreshed (e.g., hot-reload in development).

Q2. What is the difference between ContextRefreshedEvent and ApplicationReadyEvent?

ContextRefreshedEvent fires as soon as beans are ready; ApplicationReadyEvent (Spring Boot) fires later, after the embedded server is running and the app can serve requests.

Q3. How do you run code once at startup in Spring Boot?

Listen to ApplicationReadyEvent with @EventListener, or implement CommandLineRunner / ApplicationRunner, or use @PostConstruct on a bean method.

Q4. Is ContextClosedEvent guaranteed to fire on JVM crash?

No. It fires on graceful shutdown (context.close() or SIGTERM with a shutdown hook). A hard kill (SIGKILL) skips it.

Q5. Can ContextRefreshedEvent fire more than once?

Yes -- it fires on every refresh. In a standard app it fires once at startup, but in some setups (DevTools restart, test contexts) it can fire multiple times.

5 Spring Integration Overview

Channels, endpoints, and message flow

What is Spring Integration?

Spring Integration is a framework that implements **Enterprise Integration Patterns (EIP)** -- standard solutions for connecting applications, systems, and services. It models communication as **Messages** flowing through **Channels** and being processed by **Endpoints**. Think of it as internal message pipes: you put a message in at one end and it is routed, transformed, filtered, and delivered to the right handler at the other end.

Core Concepts

Concept	Description	Analogy
Message	Immutable payload + headers. The unit of data in transit.	An envelope with a letter inside
MessageChannel	Pipe between sender and receiver. Can be Direct (sync) or Queue (async).	A conveyor belt
MessageEndpoint	A component that connects to a channel to send or receive messages.	A worker at a station
Transformer	Endpoint that converts message payload from one type to another.	Translator
Router	Endpoint that decides which channel to send a message to.	Switchboard operator
Filter	Endpoint that drops messages that do not pass a predicate.	Security guard
Splitter / Aggregator	Split one message into many (or merge many into one).	Fork / join

Simple Java Example -- define a channel and flow

```
@Configuration
@EnableIntegration
public class IntegrationConfig {

    // 1. Define a channel
    @Bean
    public MessageChannel orderChannel() {
        return new DirectChannel(); // synchronous delivery
    }

    // 2. Define an integration flow
    @Bean
    public IntegrationFlow orderFlow() {
        return IntegrationFlow
            .from("orderChannel")
            .transform(String.class, String::toUpperCase)
    }
}
```

```
        .handle(msg -> System.out.println("Processed: " + msg))
        .get();
    }
}
```

Key Rules

- Add `@EnableIntegration` to a `@Configuration` class to activate Spring Integration.
- `DirectChannel` delivers messages synchronously on the sender's thread.
- `QueueChannel` delivers messages asynchronously via an internal queue.
- `IntegrationFlow` DSL is the modern way to build pipelines -- no XML required.
- Spring Integration can connect to JMS, Kafka, REST, FTP, JDBC, and many more out of the box.

Q1. What problem does Spring Integration solve?

It provides standard, reusable patterns (routing, transformation, filtering) for connecting disparate systems via messages, avoiding custom glue code.

Q2. What is a MessageChannel?

A pipe that decouples the sender from the receiver. A sender puts a message in; the channel delivers it to the subscribed endpoint.

Q3. What is the difference between DirectChannel and QueueChannel?

`DirectChannel` delivers on the sender's thread (synchronous). `QueueChannel` stores messages in a queue and a poller delivers them later (asynchronous).

Q4. What is IntegrationFlow?

A DSL (Java builder API) for defining a pipeline of channels, transformers, routers, and handlers without XML configuration.

Q5. Does Spring Integration replace Spring Messaging?

No. Spring Messaging provides the core `Message/MessageChannel` abstractions. Spring Integration builds higher-level EIP patterns on top of them.

6 @MessagingGateway & @ServiceActivator

Simple send/receive API into an Integration flow

Overview

@MessagingGateway generates a proxy for an interface: you call a regular Java method and Spring Integration converts it to a Message and sends it to a channel -- hiding all messaging code from the caller. **@ServiceActivator** marks a method as the handler for messages arriving on a channel -- it is the simplest way to connect business logic to a channel without writing any Integration infrastructure code.

Simple Java Example

```
// 1. Gateway interface -- caller knows nothing about messages
@MessagingGateway
public interface OrderGateway {
    @Gateway(requestChannel = "orderChannel")
    void sendOrder(String orderId);
}

// 2. Service activator -- handles messages on the channel
@MessageEndpoint
public class OrderProcessor {

    @ServiceActivator(inputChannel = "orderChannel")
    public void process(String orderId) {
        System.out.println("Processing order: " + orderId);
    }
}

// 3. Use the gateway like a normal service
@Service
public class ShopService {
    @Autowired OrderGateway gateway;

    public void checkout(String id) {
        gateway.sendOrder(id);    // no Message API needed
    }
}
```

Key Rules

- **@MessagingGateway** requires **@EnableIntegration** and an interface -- Spring generates the proxy.
- **@Gateway(requestChannel)** specifies which channel the method's argument is sent to.
- **@ServiceActivator(inputChannel)** links a method to the channel it listens on.
- The return type of a **@ServiceActivator** method becomes the reply message payload.
- **@MessageEndpoint** is a stereotype for **@Component** -- it is optional but communicates intent.

Q1. What does @MessagingGateway do?

It generates a Spring-managed proxy for an interface. Calling a method on the proxy automatically creates a Message and sends it to the configured channel.

Q2. Why use a gateway instead of sending messages directly?

The caller code stays clean -- it works with plain Java types, not Message objects. The messaging infrastructure is hidden behind the interface.

Q3. What is @ServiceActivator?

An annotation that marks a method as the consumer of messages arriving on a named channel. Spring Integration wires the channel to the method automatically.

Q4. Can @ServiceActivator return a reply?

Yes. If the method returns a non-void value and an outputChannel is configured (or the gateway expects a reply), the return value becomes the reply message payload.

Q5. Do I need XML to use @MessagingGateway?

No. The Java DSL (@MessagingGateway, @ServiceActivator, @EnableIntegration) is fully sufficient -- no XML is needed in modern Spring Integration.

7 Spring Scheduling

@Scheduled + @EnableScheduling: cron, fixedRate, fixedDelay

Overview

Spring's built-in scheduler lets you run methods on a repeating schedule with a single annotation. Add **@EnableScheduling** to a configuration class, then put **@Scheduled** on any void method in a managed bean. Spring uses a single-threaded scheduler by default. Three modes are available: **fixedRate**, **fixedDelay**, and **cron**.

Scheduling Modes Comparison

Mode	Annotation Example	Behaviour	When to Use
fixedRate	@Scheduled(fixedRate=5000)	Runs every 5 s from start of previous run. Overlap possible if task takes long.	Polling sensors, heartbeats
fixedDelay	@Scheduled(fixedDelay=5000)	Waits 5 s AFTER last run ends. No overlap.	Cleanup jobs, queue draining
initialDelay	@Scheduled(fixedRate=5000, initialDelay=10000)	Waits 10 s before first run, then fixedRate/Delay kicks in.	Delay startup tasks
cron	@Scheduled(cron="0 0 8 * * MON-FRI")	Runs at 08:00 every weekday. Full cron expression support.	Reports, business-hour tasks

Simple Java Example

```
// Step 1: enable scheduling
@Configuration
@EnableScheduling
public class SchedulingConfig { }

// Step 2: schedule methods
@Component
public class ReportJob {

    // Every 10 seconds (from start of previous run)
    @Scheduled(fixedRate = 10_000)
    public void heartbeat() {
        System.out.println("alive: " + System.currentTimeMillis());
    }

    // 5 seconds AFTER last run finishes
    @Scheduled(fixedDelay = 5_000)
    public void cleanup() {
        System.out.println("cleanup done");
    }

    // Every weekday at 08:00
    @Scheduled(cron = "0 0 8 * * MON-FRI")
    public void morningReport() {
```

```

        System.out.println("morning report");
    }
}

```

Cron Expression Quick Guide

Field	Values	Example
Second	0-59	0 (at second 0)
Minute	0-59	30 (at minute 30)
Hour	0-23	8 (08:xx)
Day of month	1-31 or *	* (every day)
Month	1-12 or *	* (every month)
Day of week	0-7 or MON-SUN	MON-FRI (weekdays)

Key Rules

- Add `@EnableScheduling` to exactly one `@Configuration` class to activate the scheduler.
- `fixedRate` counts from the START of the previous execution -- overlap is possible.
- `fixedDelay` counts from the END of the previous execution -- no overlap.
- Cron fields order: second, minute, hour, day-of-month, month, day-of-week.
- For parallel scheduled tasks, define a `TaskScheduler` bean with a thread pool size greater than 1.

Q1. What annotation activates Spring's task scheduler?

`@EnableScheduling` on a `@Configuration` class. Without it, `@Scheduled` methods are ignored.

Q2. What is the difference between `fixedRate` and `fixedDelay`?

`fixedRate` triggers every N ms from the start of the last run (overlap possible). `fixedDelay` waits N ms after the last run ends (no overlap).

Q3. How do you write a cron expression that runs at midnight every day?

"0 0 0 * * *" -- second 0, minute 0, hour 0, every day of month, every month, every day of week.

Q4. Does Spring's scheduler run tasks in parallel by default?

No. The default scheduler is single-threaded. If one task is slow it blocks others. Configure a `ThreadPoolTaskScheduler` bean to allow concurrent execution.

Q5. How do you delay the first run of a scheduled task?

Add `initialDelay = N` to the `@Scheduled` annotation. The task waits N milliseconds before running for the first time, then proceeds with the configured rate or delay.